1. D-ary heaps are faster than binary ones.
2. If you have parametric code, profiling is the way to tune your parameter: the best choice depends on your implementation as well as on input;
3. Be sure to run your profiling on a representative sample of your domain: if you use a limited subset, you will likely optimize for an edge case, and your results will be biased;

As a general golden rule, make sure to optimize the right thing: only profile critical code, perform high-level profiling first to spot the critical section of code whose optimization would give you the largest improvement, and that this improvement is worth the time you'll spend on it.

2.11 Summary

In this chapter we have described our first advanced data structure: the d-ary heap. We covered a number of topics, from its definition to some of the best use cases where using it will improve your code.

- **Incremental building**: the theory behind the functioning and analysis of d-way heaps makes heavy use of the basics structures and tools we described in appendices A to G. This will be true for the rest of the book, so take a look at appendices if you haven't already.
- **Concrete vs abstract data structures**: there is a difference between concrete and abstract data structures. The latter better serve to be used as black